

DataHarvest -- Rapport technique

Mathieu & Navid

Master Dev, Data & IA -- 4eme annee -- IPSSI Montpellier

31 juillet 2026

1. Introduction et perimetre

1.0 Probleme resolu et limites de portee

Quel probleme DataHarvest resout-il ? Scraper un nouveau site HTML statique demande habituellement d'ecrire un script dedie (requetes, parsing, pagination, stockage, gestion d'erreurs) a chaque fois. DataHarvest remplace ce script par un moteur generique unique, pilote entierement par un fichier de configuration YAML (URL, selecteurs CSS, pagination, backend de stockage) -- exactement l'objectif pose par le sujet section 1 : *"scraper n'importe quel site HTML statique simplement en modifiant un fichier de configuration -- sans toucher au code source."* Concretement, ajouter un 6e site au projet ne demande qu'un nouveau fichier `configs/site6.yaml` ; aucune ligne de `dataharvest/*.py` n'a change entre le premier et le cinquieme site.

Limites de portee, assumees des le depart :

- **Pas de rendu JavaScript.** DataHarvest ne fait que `requests.Session().get()` -- aucun moteur de rendu. Deux sites envisages (legifrance.gouv.fr, numerama.com) ont ete ecartes pour cette raison precise, verifiee et non supposee (section 5.1).
- **Synchrone, pas de parallelisme.** Une requete a la fois, jamais de pool de connexions concurrentes (voir comparaison avec Scrapy, section 3.3).
- **Pas de contournement anti-bot.** Ni rotation de User-Agent, ni pool de proxies -- choix ethique deliberelement, pas juste une omission (section 3.1 et section 4).
- **Un schema d'item par site.** Chaque config extrait un seul type d'objet (ex: soit des livres, soit des citations) ; pas de scraping relationnel/imbrique (ex: liste d'articles + commentaires de chaque article en une seule config).
- **Selecteurs CSS par champ, pas de decouverte automatique de structure.** L'utilisateur doit fournir les selecteurs -- DataHarvest ne devine pas la structure d'une page inconnue.

1.1 Sites retenus pour les tests

5 sites ont ete retenus, couvrant 4 niveaux de difficulte differents parmi les 4 proposes par le sujet (qui exige au minimum 2) :

Site	Niveau	Champs cibles
books.toscrape.com	1	titre, url, prix, disponibilite, note
quotes.toscrape.com	1	texte, auteur, tags
fr.wikipedia.org	2	wikitable ET infobox (listes, classements)
blogdumoderateur.com	3	titre, date, categorie
news.ycombinator.com	4	titre, url, score, domaine source, commentaires

2. Architecture et choix de conception

2.0 Composants decouplés, YAML, injection de dependances

Pourquoi une architecture en composants decouplés plutôt qu'un script monolithique ? Le gain concret le plus visible dans ce projet n'est pas théorique : sur les 76 tests de la suite, un seul (`test_integration.py`) ouvre une vraie connexion réseau -- les 75 autres testent chaque composant (`Fetcher`, `Pipeline`, `Validator`, `Store`, `Orchestrator`) isolément, avec des mocks ou du HTML statique. Ça n'aurait pas été possible avec un script unique où `fetch/parse/valide/stocke` seraient entremêlés : impossible de tester "est-ce que `RetryMiddleware` calcule le bon délai" sans également déclencher une vraie requête HTTP. Le découplage a aussi permis un vrai parallélisme de travail : Navid a écrit `Config/Validator/Store` en même temps que `Fetcher/Pipeline` étaient écrits de l'autre côté, sans qu'aucun des deux ne bloque l'autre -- chacun ne dépend que d'une interface (le dict retourne par `Pipeline.process()`, le tuple retourne par `Validator.validate()`), jamais de l'implémentation interne du composant voisin. Côté extensibilité, le mode `item_selector` de `GenericPipeline` (section 2.2) a été ajouté sans toucher une seule ligne de `Fetcher`, `Validator`, `Store` ou `Orchestrator`.

Pourquoi le chargement de configuration via YAML plutôt que des arguments CLI ? Quand préférer l'un ou l'autre ? Une config de site a 5 blocs imbriqués (`url`, `pagination`, `selectors` -- un dict a nombre de clés variable selon le site, de 3 à 5 champs -- `fetcher`, `store`, et optionnellement `validator/item_selector`). En arguments CLI, ça donnerait des commandes du type `--selector-titre=... --selector-url=... --selector-prix=... --pagination-pattern=... --pagination-max-pages=...` -- illisible, et surtout impossible à versionner proprement dans Git (5 fichiers `configs/*.yaml` diffables, relisibles, partageables ; pas 5 lignes de commande à se rappeler ou copier-coller). À l'inverse, le CLI garde des arguments simples pour des choix ponctuels à l'exécution : `--dry-run` (comportement différent, pas une donnée structurée du site) ou `--config` (juste un chemin de fichier). La règle suivie ici : la structure récurrente et versionnée va en YAML, les choix d'exécution ponctuels vont en CLI.

Pourquoi l'injection de dépendances dans le constructeur plutôt que des imports directs ?

Exemple concret. `Fetcher(config, middlewares=[...])` reçoit sa liste de middlewares au lieu de les importer et instancier lui-même. Ça se voit directement dans `tests/test_orchestrator.py` : `patch.object(orchestrator.fetcher.session, "get", ...)` intercepte les appels réseau en remplaçant `session.get` sur l'instance `requests.Session` déjà construite et exposée par `Fetcher` -- possible uniquement parce que `Fetcher` est un attribut public, injectable et inspectable de `Orchestrator`, pas un appel `requests.get(...)` caché à l'intérieur d'une méthode. Si `Orchestrator` avait importé et appelé `requests` directement, le seul moyen de le tester aurait été de monkey-patcher la bibliothèque `requests` globalement -- fragile, et risque de fuite entre tests exécutés en parallèle. L'injection isole chaque test à l'instance qu'il construit lui-même.

2.1 Pourquoi le pattern middleware pour le Fetcher, et gestion du retry

Le retry est délibérément scindé en deux responsabilités, réparties de part et d'autre du pattern middleware :

- `RetryMiddleware` (`middleware.py`) decide si une reponse ou une exception doit declencher un nouvel essai (statuts 408/429/5xx, exceptions reseau) et calcule le *delai a respecter* -- l'en-tete HTTP `Retry-After` renvoie par le serveur quand il est present (typiquement sur un 429), sinon un backoff exponentiel `base_delay * 2^attempt`.
- `Fetcher.fetch()` reste seul responsable de la boucle elle-meme : comptage des tentatives, `time.sleep()`, abandon final via `FetchError`.

Sans ce pattern, la politique de retry (quels codes retenter, comment lire `Retry-After`) serait codee en dur dans `Fetcher`, qui deviendrait responsable a la fois du transport HTTP et de la decision de retry -- impossible a tester ou remplacer independamment. Avec le pattern, `RetryMiddleware.process_response()` se teste seul (statut -> exception levee ou non, delai calcule) sans jamais ouvrir de connexion reseau.

Deux delais independants coexistent dans le code et ne doivent pas etre confondus :

Parametre	Source	Role
<code>config.fetcher.delay</code>	YAML, cle obligatoire validee par <code>Config</code>	Espacement entre deux URLs differentes, dans <code>fetch_all()</code>
<code>RetryMiddleware.base_delay</code>	Default Python (<code>1.0</code>), jamais lu depuis le YAML	Base du backoff exponentiel entre deux tentatives sur la meme URL

Le sujet (section 4.2) ne demande de rendre configurable que `max_retries` pour `RetryMiddleware` -- fait via `config.fetcher.retries` -- et non le `base_delay` du backoff, qui reste donc un default de code plutot qu'une cle YAML.

Verifications effectuees (via `unittest.mock`, sans dependre du reseau reel) :

- 429 avec en-tete `Retry-After: 2` -> le delai fourni par le serveur est respecte tel quel, meme avec un `base_delay` de calcul different (5.0s) : le backoff calcule n'est pas utilise dans ce cas.
- 500 sans `Retry-After` -> retombe sur le backoff exponentiel (`0.01s`, puis `0.02s` sur les tentatives suivantes).
- Exception reseau (`requests.ConnectionError`) -> meme comportement de backoff exponentiel, avec le type d'exception journalise a chaque tentative.
- 404 -> `FetchError` levee immediatement, sans passer par la boucle de retry (statut definitif, hors `RETRYABLE_STATUS_CODES`).

2.2 Pourquoi BasePipeline est une ABC, et flat vs scope dans GenericPipeline

`BasePipeline` est une classe abstraite (`abc.ABC` avec `@abstractmethod`) plutot qu'une classe normale a methodes vides, pour une raison concrete et pas seulement stylistique : Python leve une `TypeError` a l'instanciation si `process()` ou `next_page_url()` n'est pas implementee, *avant meme* que le code tourne. Avec une classe normale a methodes vides (`def process(self): pass`), une pipeline concrete qui oublierait d'implementer `process()` heriterait silencieusement de la version vide -- elle renverrait `None` ou `[]` sans erreur, et le bug ne serait decouvert qu'en production, quand l'Orchestrator recevrait

des donnees vides sans explication. L'ABC deplace l'erreur du runtime tardif au chargement du module : c'est une garantie donnee a quiconque ecrit une pipeline personnalisee, pas juste une convention documentee dans un commentaire.

Les deux implementations concretes, `GenericPipeline` et `PaginationPipeline`, illustrent aussi un choix de conception fait pendant le developpement plutot qu'anticipe des le depart. Le modele par default ("flat" : un selecteur CSS par champ applique a toute la page, items reconstruits en zippant les correspondances par position) est simple et suffit pour la majorite des sites testes (books.toscrape.com, quotes.toscrape.com, blogdumoderateur.com). Mais il echoue silencieusement -- sans exception, juste des donnees fausses -- des qu'un champ est present pour certains items et absent pour d'autres. Trouve concretement sur news.ycombinator.com : le domaine source (`span.sitestr`) n'existe que pour les stories avec lien externe (absent des posts "Ask HN"), ce qui volait le domaine d'une story a l'autre. Plutot que d'imposer partout le modele plus lourd (conteneur d'item scope, comme le fait Scrapy nativement), le mode flat est reste le default et un second mode optionnel (`item_selector`, chaque champ cherche dans son propre conteneur) n'a ete ajoute que la ou un site le demande reellement -- 1 site sur 5 (news.ycombinator.com) en a effectivement besoin. Illustration concrete du principe "ne pas batir la complexite avant d'en avoir la preuve".

2.3 Orchestrator : stockage par lot de pages et rapport de session

`Orchestrator.run()` sauvegarde les items valides page par page (`store.save(valides)` a chaque iteration de la boucle de pagination), plutot que d'accumuler tous les items de toutes les pages en memoire puis de sauvegarder en une seule fois a la fin. Deux raisons concretes : (1) si le scraping s'interrompt en cours de route (erreur reseau irrecuperable au milieu des `max_pages` pages), les pages deja traitees restent stockees au lieu d'etre perdues -- comportement explicitement demande par le sujet ("appeler store.save() par lot de pages") ; (2) ca borne l'empreinte memoire a une page a la fois plutot qu'a l'integralite du site, ce qui compte sur les sites a forte pagination (jusqu'a 50 pages sur books.toscrape.com).

Le rapport de session retourne par `run()` est un simple dict (pas une classe dediee) avec exactement les 6 cles demandees par le sujet -- `pages_scrapees`, `items_trouves`, `items_valides`, `items_rejetes`, `items_stockes`, `duree_secondes` -- construit par une methode privree `_build_report()` separee de la boucle principale : la logique de comptage (boucle de pagination) et la logique de mise en forme (rapport) restent independantes, chacune testable seule.

3. Comparaison avec Scrapy

3.1 Fonctionnalités que Scrapy offre et que DataHarvest n'offre pas

1. Moteur asynchrone (Twisted, `CONCURRENT_REQUESTS`)

Scrapy traite plusieurs requêtes en parallèle sans bloquer le thread principal. DataHarvest est volontairement synchrone (un seul thread, `requests.Session` bloquant) : implémenter un moteur asynchrone complet (event loop, ordonnancement des coroutines, backpressure) dépasse largement le budget de 5h et n'était pas l'objectif pédagogique du projet, qui porte sur l'architecture par composants et non sur la réimplémentation d'un moteur réseau asynchrone.

2. Cache HTTP disque (`HttpCacheMiddleware`)

Permet de rejouer un scraping sans re-solliciter le réseau, utile en développement pour itérer rapidement sur les sélecteurs. Non implémenté : bon candidat d'évolution future (voir section 6), mais ajoute de la complexité (invalidation, sérialisation) hors scope du sujet initial.

3. Rotation de User-Agent / pool de proxies (`scrapy-user-agents`, `scrapy-rotating-proxies`)

Écarte volontairement, pas seulement par manque de temps : ces mécanismes servent à contourner la détection anti-bot des sites, ce qui contredit l'exigence d'un User-Agent identifiable posée en section 4 (éthique) et l'esprit "bon citoyen du web" du projet. Notre Fetcher utilise au contraire un unique User-Agent identifiable, fourni par la configuration.

4. Rendu JavaScript (`scrapy-playwright`, `Splash`)

DataHarvest cible explicitement le HTML statique (cf. périmètre, section 1). Ajouter un moteur de rendu JS changerait la nature de l'outil et sortirait du périmètre défini par le sujet.

5. Export "feed" déclaratif multi-format en une option CLI (`-o out.json`)

Scrapy écrit vers plusieurs formats simultanément sans code supplémentaire. `Store.export_to()` couvre notre besoin de conversion entre backends (csv/sqlite/json), mais pas l'écriture simultanée vers N formats en une seule commande : complexité jugée non prioritaire vu le temps imparti.

3.2 Dans quels cas préférer DataHarvest à Scrapy ?

Scenario 1 -- intégration dans une application existante. Un développeur qui a déjà un script Python (ETL, notebook, tâche cron) et veut ajouter un scraping ponctuel sans faire entrer Twisted, et son modèle de programmation asynchrone spécifique, dans ses dépendances peut importer directement `Orchestrator` comme une classe Python normale et bloquante, sans reacteur à démarrer ni arrêter.

Scenario 2 -- contexte pédagogique ou script jetable. Pour scraper cinq pages une seule fois, ou dans un cadre étudiant où l'objectif est de comprendre chaque étape de la chaîne (fetch -> parse -> valider -> stocker), un outil d'une dizaine de fichiers lisibles de bout en bout en quelques minutes est préférable à l'apprentissage de toute la surface de configuration de Scrapy (`settings.py`, signals, items, spiders, middlewares empilés).

3.3 Impact synchrone vs asynchrone -- calcul théorique (100 pages, `DOWNLOAD_DELAY=1s`)

Hypotheses : delai configure de 1s entre requetes, temps moyen reseau + traitement par page $t_{req} \approx 0.3s$, Scrapy avec sa concurrence par default `CONCURRENT_REQUESTS_PER_DOMAIN = 8`.

DataHarvest (synchrone). Chaque requete attend la fin de la precedente :

```
temps_total ≈ N × (delay + t_req) = 100 × (1 + 0.3) = 130 s (~2 min 10)
```

Scrapy (asynchrone, concurrence C = 8). Jusqu'a 8 requetes en vol simultanement, le delai s'applique par slot de concurrence et non de facon strictement sequentielle :

```
temps_total ≈ (N / C) × (delay + t_req) = (100 / 8) × 1.3 ≈ 16,25 s
```

Soit un gain theorique d'environ **8x** pour Scrapy sur ce volume, l'ecart se creusant encore a mesure que le nombre de pages augmente. A l'echelle de ce projet (5 sites de quelques dizaines de pages chacun), l'ecart reste de l'ordre de quelques dizaines de secondes contre quelques secondes -- non bloquant pour notre usage -- mais deviendrait significatif au-dela de quelques centaines de pages ou de plusieurs domaines scrapes en parallele.

4. Cadre legal et ethique

4.1 Analyse des robots.txt (verifie le 31/07/2026, curl reel sur chaque site)

Site	robots.txt	Nos URL concernees	Respecte ?
books.toscrape.com	Absent (404)	/, /catalogue/page-N.html	Pas de restriction declaree
quotes.toscrape.com	Absent (404)	/, /page/N/	Idem
fr.wikipedia.org	Present. Disallow: /w/, /api/, /wiki/Special:... pour User-agent: *	/wiki/Liste_des_presidents_hors_des_republique_francaise	Oui -- hors des chemins interdits
blogdumoderateur.com	Present. Disallow: /wp-admin, /feed/, /comments, /*.php\$...	/ (page d'accueil)	Oui -- la racine n'est pas listee
news.ycombinator.com	Present. Disallow: /login, /vote?, /reply?... + Crawl-delay: 30	/, /?p=2	Chemins : oui. Delai : non au depart -- voir ci-dessous

Le point le plus important trouve ici : `news.ycombinator.com` declare `Crawl-delay: 30` (30 secondes entre requetes) pour `User-agent: *`. Notre config utilisait `delay: 1.0` -- un vrai manquement, pas volontaire. En creusant pour le corriger, un second bug plus large est apparu : `Orchestrator.run()` n'appelait `jamais time.sleep()` entre deux pages de la meme pagination -- `config.fetcher.delay` n'etait utilise que dans `Fetcher.fetch_all()`, une methode que `Orchestrator` n'appelle pas (il fait ses propres appels a `fetch()` un par un dans sa boucle de pagination). Donc meme avec `delay: 30.0` dans le YAML, rien ne se serait passe. Corrige aux deux niveaux : `configs/hackernews.yaml` passe a `delay: 30.0`, et `Orchestrator.run()` fait maintenant `time.sleep(self.config.fetcher.delay)` entre deux pages. Verifie en conditions reelles : le crawl HN complet (2 pages) est passe de ~1s a ~32s apres correction.

4.2 Base legale RGPD et donnees personnelles

Base legale (Art. 6 RGPD). Les donnees collectees ici (titres de livres, citations, metadonnees d'articles/posts) relevent de l'**interet legitime** (Art. 6.1.f) : collecte a des fins pedagogiques, sur des donnees deja publiees publiquement, sans profilage ni decision automatisee affectant une personne, volume limite (quelques dizaines a centaines d'items par site), et le RGPD ne s'applique de toute facon qu'aux donnees a caractere personnel -- la grande majorite des champs collectes (prix, disponibilite, note, score, tags, categorie) n'en sont pas.

Les donnees collectees sont-elles des donnees personnelles ? Cas par cas :

- books.toscrape.com, quotes.toscrape.com (champ `tags`), news.ycombinator.com (`domaine`, `score`) : aucune donnee personnelle -- ce sont des metadonnees de contenu, pas des identifiants de personnes.
- quotes.toscrape.com, champ `auteur` : des noms de personnes (ex: "Albert Einstein"), mais toutes deceduees -- le RGPD ne protege que les personnes physiques vivantes (considerant 27). Non concerne.
- fr.wikipedia.org : des noms de presidents de la Republique francaise, personnages publics par definition de leur fonction, deja publies sous leur identite reelle par eux-memes/par l'Etat. Argument "personnage public + interet historique/documentaire" applicable ; le cas serait different pour un individu prive.
- news.ycombinator.com, champ `titre/url` : peuvent parfois contenir un nom si le titre d'un post en cite un, mais aucun champ scrape ne cible directement un identifiant de personne (on ne recupere pas le pseudo de l'auteur du post, volontairement absent de nos selecteurs).

4.3 Mecanismes techniques de "bon citoyen" du web

- **Throttling** : `config.fetcher.delay` entre chaque page (desormais reellement applique par `Orchestrator`, voir 4.1), backoff exponentiel sur erreurs (`RetryMiddleware`), respect de l'en-tete `Retry-After` quand le serveur le fournit, et prise en compte du `Crawl-delay` propre a chaque site quand robots.txt en declare un.
- **User-Agent identifiable** : toujours `DataHarvest/1.0 (+contact@ipssi.fr)`, jamais le default `python-requests/x.x` ni un User-Agent de navigateur usurpe -- rotation de UA explicitement exclue (section 3.1) car ca sert a contourner la detection, pas a etre identifiable.
- **UNIQUE(url) en sqlite** : `INSERT OR IGNORE` evite de re-stocker (et donc de re-scrapier inutilement pour re-tester) un item deja recupere.
- **Retries bornes** : `max_retries` configurable, jamais de boucle infinie sur un site indisponible.
- **Respect des chemins interdits** : aucune des 5 configs ne cible une URL listee dans un `Disallow` (verifie section 4.1).

5. Difficultes et retrospective

5.1 Difficultes rencontrees sur Pipeline

- **Desalignement silencieux en mode flat** : le piege le plus subtil rencontre pendant le developpement. Sur l'infobox Wikipedia, un selecteur naif (`table.infobox tr td`) recuperait la valeur de la ligne d'en-tete (un `<td colspan="2">` sans `<th>` associe) au lieu de la premiere vraie ligne de donnees, decalant tout le reste d'un cran -- aucune exception levee, juste des donnees fausses. Corrige avec le combinateur CSS `+` (`th[scope='row'] + td`) pour forcer l'appariement ligne par ligne. Le meme type de bug a ete retrouve independamment sur `news.ycombinator.com` (domaine source absent de certains items), corrige cette fois via le nouveau mode `item_selector` (voir section 2.2).
- **Attributs `class` multi-valeurs** : sur `books.toscrape.com`, la note (`p.star-rating.Three`) encode l'information utile dans un deuxieme mot de l'attribut `class`, que BeautifulSoup retourne comme une liste Python plutot qu'une chaine. Le pipeline la rejoint en une seule chaine ("`star-rating Three`") mais n'isole pas le mot utile ("Three") -- limite assumee et documentee dans les tests, pas corrige (voir section 6 pour une piste).
- **Sites non scrapables statiquement** : `legifrance.gouv.fr` et `numerama.com` ont ete abandonnes apres verification reelle (curl sur le vrai HTML, pas une supposition) plutot qu'ecarter a priori.
- `legifrance.gouv.fr` : sa page de recherche est une SPA Angular -- le HTML brut recupere par `requests` ne contient qu'un message `<noscript>Javascript est desactive...</noscript>`, aucun contenu exploitable sans executer le JavaScript. Des pages individuelles (`/codes/article_lc/...`) sont, elles, bien rendues cote serveur -- mais scraper une liste de resultats de recherche generique n'est pas possible avec l'approche statique de DataHarvest.
- `numerama.com` : le sujet demande "titre, date, tags, nombre de commentaires". Verifie a la fois sur la page d'accueil ET sur une vraie page d'article (curl, sans JS) : aucun tag ni compteur de commentaires n'est present dans le HTML statique -- pas de JSON-LD, pas de widget Disqus/Coral cote serveur, probablement charges en JavaScript cote client si meme disponibles. Seuls titre/url/date auraient ete exploitables pour ce site avec l'approche statique de DataHarvest, ce qui ne couvre pas les champs demandes par le sujet.
- **Bug d'integration entre composants ecrits separement** : `Store.__init__` gardait le parametre `path` tel quel au lieu de le convertir en `Path`, alors que `save_csv/save_json` appellent `self.path.exists()` -- ca plantait des que `Store` recevait une simple string (le cas de `config.store.path`, venu du YAML). Corrige avec `self.path = Path(path)`, plus `self.path.parent.mkdir(parents=True, exist_ok=True)` (necessaire aussi car `output/` est gitignore, donc absent sur un clone frais).

5.2 Application sur les 5 sites : ce que les tests unitaires n'avaient pas vu

Les 5 configs (`configs/*.yaml`) ont ete testees en conditions reelles via `python -m dataharvest crawl --config configs/siteN.yaml` (vrai reseau, pas de mock) :

Site	Pages	Items stockes
------	-------	---------------

books.toscrape.com	2	40/40
quotes.toscrape.com	2	20/20
fr.wikipedia.org	1	49 (25 rejetees proprement)
blogdumoderateur.com	1	44/44
news.ycombinator.com	2	60/60

Deux constats concrets qui n'etaient pas visibles avant ce test de bout en bout :

- **required_fields/item_selector ne peuvent pas rester en dur dans Orchestrator** : le pseudo-code du sujet fixe `Validator(required_fields=['titre', 'url'])`, mais seuls 2 des 5 sites reels ont naturellement ces deux champs. Sans changement, `items_stockes` aurait ete 0 pour quotes.toscrape.com (aucun item n'a de champ `url`). Rendu configurable par site via un bloc YAML optionnel (`validator: required_fields: [...], item_selector: "..."`), avec repli sur le comportement impose par default si le site n'en a pas besoin -- books.toscrape.com et news.ycombinator.com continuent d'utiliser `['titre', 'url']` sans rien declarer de plus.
- **Un selecteur teste en unitaire peut rester faux contre le vrai site** : le selecteur `commentaires` de news.ycombinator.com (`.subline a:last-child`) recuperait "X heures" au lieu du nombre de commentaires. En cause : `<span class="age"><a>X heures</a></span>` -- ce lien est le dernier (et seul) enfant de `span.age`, donc il matche aussi `:last-child`, et il precede le vrai lien commentaires dans le DOM ; `select_one()` renvoie le premier trouve. Le fixture de test (`tests/test_pipeline_real_sites.py`) etait simplifie et ne contenait pas ce lien imbrique -- il passait donc alors que le vrai site echouait. Corrige avec `.subline > a:last-child` (enfant direct), et le fixture de test corrige en meme temps pour refleter la vraie structure HTML. Argument concret pour le test d'integration reseau reel (section 5 du sujet) : un test unitaire n'est fiable que si son fixture est fidele au HTML reel.
- **fr.wikipedia.org** : la page a en realite 2 tables `class="wikitable"` (la liste des presidents, et un tableau de resultats electoraux en %). Nos selecteurs etant globaux a toute la page, `annee` matche des lignes des deux tables, mais `nom/url` (qui exigent un lien) ne matchent que celles du tableau des presidents. Il y a donc plus de valeurs `annee` que de `nom/url` : les valeurs en trop se retrouvent assemblees dans des items sans `nom` ni `url` (ex: `{'nom': '', 'url': '', 'annee': '89 %'}`). Pas grave ici : `required_fields: [nom]` rejette proprement ces 25 items. Encore une illustration des limites du mode "flat" (section 2.2).
- **config.fetcher.delay jamais applique dans la boucle de pagination** : trouve en verifiant le `Crawl-delay: 30` de news.ycombinator.com (section 4.1). `Orchestrator.run()` appelait `Fetcher.fetch()` page par page sans jamais attendre entre deux pages -- le delai configure n'etait utilise que dans `Fetcher.fetch_all()`, une methode jamais appelee par `Orchestrator`. Un bug qui ne se voyait dans aucun test (tous utilisent `delay: 0.0` pour rester rapides) et qui n'a ete remarque qu'en cherchant a etre conforme a un robots.txt reel. Corrige : `time.sleep(self.config.fetcher.delay)` entre deux pages quand il y en a une suivante.

5.3 Retrospective

Difficulté technique principale. Le désalignement silencieux du mode "flat" (section 2.2, 5.1) : aucune exception n'est levée, les données sont juste fausses. Contrairement à une erreur qui plante et se voit immédiatement, ce type de bug ne se remarque qu'en inspectant manuellement le contenu scrape -- il est apparu indépendamment sur deux sites complètement différents (l'infobox Wikipedia et news.ycombinator.com), ce qui suggère que c'est une limite structurelle du modèle plutôt qu'un accident isolé.

Ce qu'on referait différemment. Vérifier systématiquement le vrai HTML (curl) et le vrai robots.txt de chaque site *avant* d'écrire selecteurs et config, plutôt qu'après. Plusieurs correctifs de ce rapport (les vraies classes CSS de blogdumoderateur.com, différentes de l'exemple du sujet lui-même qui date probablement d'une version antérieure du site ; le `Crawl-delay` de news.ycombinator.com ; le bug de délai jamais appliqué) ont été trouvés tardivement parce que la vérification s'est faite par étapes plutôt qu'en un seul passage systématique au début du projet.

Fonctionnalité non implémentée par manque de temps. Isoler le mot utile d'un attribut `class` multi-valeurs (ex: `"star-rating Three"` -> `"Three"` sur books.toscrape.com). S'implémenterait comme une petite extension de la syntaxe `::attr(...)` déjà existante, par exemple un filtre supplémentaire du style `::attr(class)|last-word`, ou un registre de fonctions de transformation nommées passées au constructeur de `GenericPipeline`. Également non fait : `HttpCacheMiddleware` (section 3.1) et l'export multi-format simultané.

Repartition des tâches. Visible dans `git shortlog -sn --all` et dans les messages de commits :

- **Navid** : `Config` (chargement YAML/JSON, validation), `Validator` (champs requis, URL, longueur min), `Store` (csv/sqlite/json, `export_to()`), le squelette CLI (`app.py`, parsing `argparse`), `.gitignore`, une partie du README et des tests de couverture.
- **Mathieu** : `BaseMiddleware/LoggingMiddleware/RetryMiddleware`, `Fetcher`, `GenericPipeline/PaginationPipeline`, `Orchestrator`, le câblage final du CLI à `Orchestrator` (le `command_crawl` de Navid avait un TODO en attente), les 5 configs de sites et leur vérification en conditions réelles, la majorité de ce rapport technique.
- Repartition assez proche de la suggestion du planning du sujet (section 9), avec des écarts : Navid a aussi fait `Store` (prévu sans attribution dans le planning), et le CLI a été commencé par Navid puis terminé par Mathieu une fois `Orchestrator` disponible -- une dépendance réelle entre les deux parties du planning ("Sprint 4"), pas un découpage parfaitement étanche.

6. Perspectives

6.1 Une 6e brique : Notifier

Interface calquee sur le pattern deja utilise pour les middlewares (ABC + injection au constructeur) :

```
from abc import ABC, abstractmethod class BaseNotifier(ABC): @abstractmethod def
notify(self, report: dict) -> None: """Recoit le rapport de session (memes cles que
Orchestrator.run()).""" class SlackNotifier(BaseNotifier): def __init__(self,
webhook_url: str): self.webhook_url = webhook_url def notify(self, report: dict) ->
None: message = ( f"Scraping termine : {report['items_stockes']} items stockes " f"sur
{report['pages_scrapees']} pages en {report['duree_secondes']:.1f}s "
f"({report['items_rejetes']} rejetes)" ) requests.post(self.webhook_url, json={"text":
message}) class EmailNotifier(BaseNotifier): def __init__(self, smtp_config: dict,
destinataire: str): ... def notify(self, report: dict) -> None: ...
```

`Orchestrator.__init__` accepterait un parametre optionnel `notifiers: list[BaseNotifier] = None`, et `run()` appellerait `notifier.notify(report)` sur chacun juste avant de retourner -- utile des que le scraping tourne sans supervision (cron, scheduler), pour etre alerte en cas de chute soudaine du nombre d'items stockes.

6.2 Sites JavaScript sans Selenium

Remplacer `Fetcher` par une implementation basee sur `Playwright` (moteur headless Chromium/Firefox/WebKit, API plus moderne et plus rapide que Selenium/WebDriver). Le point cle : ni `GenericPipeline` ni `Validator` ni `Store` n'auraient besoin de changer, puisqu'ils ne consomment qu'une chaine HTML brute produite par `Fetcher.fetch()`. Un `PlaywrightFetcher` implementerait la meme methode `fetch(url) -> str`, mais en interne ferait `page.goto(url)` puis retournerait `page.content()` (le DOM apres execution du JS) au lieu du texte brut de la reponse HTTP. Le choix du moteur deviendrait une simple cle de config (`fetcher_type: requests` ou `playwright`), sans toucher au reste de la chaine -- exactement le genre d'extension que permet l'architecture decouplee (section 2.0).

6.3 Distribution sur PyPI

- Un `pyproject.toml` (metadata, dependances en plages de versions plutot qu'epinglees en dur, backend de build).
- Un point d'entree `console_scripts` (`dataharvest = dataharvest.app:main`) pour que la commande `dataharvest` soit disponible directement apres `pip install`, sans passer par `python -m`.
- Une licence explicite (`LICENSE`).
- Des docstrings et type hints complets sur l'API publique (aujourd'hui presents par endroits, pas systematiques).
- Une gestion de version semantique (`__version__` actuellement fige a `"1.0.0"`, jamais incremente).
- Un pipeline CI (lint + tests + publication automatique sur tag, via GitHub Actions + `twine`).

Bibliographie

- Documentation Scrapy -- Architecture overview, RetryMiddleware, HttpCacheMiddleware, AutoThrottle : <https://docs.scrapy.org/>
- Documentation requests / urllib3 -- gestion des sessions et de la decompression HTTP.
- RFC 9309 -- Robots Exclusion Protocol (robots.txt), et RFC 9110 section 10.2.3 -- l'en-tete HTTP Retry-After.
- Reglement (UE) 2016/679 (RGPD) -- Article 6 (bases legales du traitement) et considerant 27 (donnees des personnes decedees hors champ d'application).
- robots.txt reels consultes le 31/07/2026 : books.toscrape.com, quotes.toscrape.com, fr.wikipedia.org, blogdumoderateur.com, news.ycombinator.com.
- Documentation Playwright -- <https://playwright.dev/python/> (mentionnee section 6.2).